**МИНОБРНАУКИ РОССИИ**
**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ**
**ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**
**«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)**
**Кафедра МО ЭВМ**

# ОТЧЕТ

**по лабораторной работе №5**

**по дисциплине «Построение и анализ алгоритмов»**

**Тема: Ахо-Корасик**

**Вариант: 7**

Студентка гр. 4343 _____ Попова Е.П.

Преподаватель _____ Жангиров Т.Р.

Санкт-Петербург

2026

**Задание.**

6.1 Ахо-Корасик

Разработайте программу, решающую задачу точного поиска набора образцов.

Вход:

Первая строка содержит текст (T,$1 \leq |T| \leq 100000$ ).

Вторая - число n ($1 \leq n \leq 3000$), каждая следующая из n строк содержит шаблон из набора $P=\{p1,\ldots,pn\}$ $1 \leq |pi| \leq 75$. Все строки содержат символы из алфавита {A,C,G,T,N}

Выход:

Все вхождения образцов из P в T.

Каждое вхождение образца в текст представить в виде двух чисел - i p

Где i - позиция в тексте (нумерация начинается с 1), с которой начинается вхождение образца с номером p (нумерация образцов начинается с 1).

Строки выхода должны быть отсортированы по возрастанию, сначала номера позиции, затем номера шаблона.

6.2

Используя реализацию точного множественного поиска, решите задачу точного поиска для одного образца с джокером.

В шаблоне встречается специальный символ, именуемый джокером (wild card), который "совпадает" с любым символом. По заданному содержащему шаблоны образцу P необходимо найти все вхождения P в текст T.

Например, образец ab??c? с джокером ? встречается дважды в тексте xabvccbababcax.

Символ джокер не входит в алфавит, символы которого используются в T. Каждый джокер соответствует одному символу, а не подстроке неопределённой длины. В шаблон входит хотя бы один символ не джокер, т.е. шаблоны вида ??? недопустимы. Все строки содержат символы из алфавита {A,C,G,T,N}.

Вход:

Текст (T,$1 \leq |T| \leq 100000$ )

Шаблон (P,$1 \leq |P| \leq 40$)

Символ джокера

Выход:

Строки с номерами позиций вхождений шаблона (каждая строка содержит только один номер).

Номера должны выводиться в порядке возрастания.

**Выполнение работы.**

Основные структуры данных

6.1 :

- text (str) – текст, в котором выполняется поиск образцов.
- n (int) – количество образцов для поиска.
- patterns (list[str]) – список образцов для поиска.
- Node (class) – вершина бора (автомата).
    - children (dict) – словарь переходов по символам.
    - suffix_link (Node) – суффиксная ссылка на наибольший собственный суффикс, являющийся префиксом какого-либо образца.
    - output_link (Node) – терминальная (конечная) ссылка на ближайшую вершину по суффиксным ссылкам, в которой заканчивается хотя бы один образец.
    - patterns (list[int]) – номера образцов, заканчивающихся в данной вершине.
    - depth (int) – глубина вершины в боре (длина префикса).
    - id (int) – уникальный идентификатор вершины.
- root (Node) – корневая вершина бора (начальное состояние автомата).
- node_count (int) – общее количество вершин в автомате.
- results (list[tuple[int, int]]) – список найденных вхождений в формате (позиция, номер_образца).

- queue (deque) – очередь для обхода автомата в ширину (BFS) при построении суффиксных ссылок.

6.2:

- text (str) – текст для поиска.
- pattern (str) – шаблон с джокерами.
- wildcard (str) – символ джокера (ровно один символ).
- n (int) – длина текста.
- m (int) – длина шаблона.
- segments_info (list[tuple[int, str]]) – список сегментов шаблона в виде пар (отступ от начала шаблона, строка сегмента).
- seg_checks (list[tuple[int, int]]) – список пар (отступ, длина сегмента), используется при подсчёте голосов.
- C (list[int]) – массив счётчиков: C[i] показывает, сколько сегментов шаблона совпало, если бы шаблон начинался в позиции i.
- results (list[int]) – список позиций найденных вхождений (1-based).
- Node (class) – вершина бора:
    - children (dict) – словарь переходов;
    - suffix_link (Node) – суффиксная ссылка;
    - output_link (Node) – сжатая суффиксная (выходная) ссылка на ближайшую терминальную вершину;
    - segment_ids (list[int]) – номера сегментов, заканчивающихся в данной вершине;
    - depth (int) – глубина вершины.
- AhoCorasick (class) – автомат:
    - root (Node) – корневая вершина;
    - segments (list[str]) – список строк-сегментов.

Вариант 7 (Графический интерфейс на CustomTkinter):

В файле gui_5.py объединены оба алгоритма. Чтобы классы 6.1 и 6.2 не конфликтовали по именам, классы задачи 6.2 получили префикс Wildcard.

Алгоритмическая часть:

- WildcardNode (class) – вершина бора для задачи с джокером (аналог Node из 6_2.py, дополненный полем id для визуализации).
- WildcardAhoCorasick (class) – автомат сегментов (аналог AhoCorasick из 6_2.py):
    - root (WildcardNode) – корневая вершина;
    - segments (list[str]) – строки сегментов;
    - segments_info (list[tuple[int, str]]) – информация о сегментах;
    - node_count (int) – количество вершин;
    - nodes_by_id (dict) – доступ к вершинам по id для визуализации.

Интерфейсная часть:

- KmpApp (class) – главный класс приложения, наследующий ctk.CTk.
- tabview (ctk.CTkTabview) – контейнер с тремя вкладками: «Ахо-Корасик», «Поиск с джокером», «Визуализация автомата».
- entry_text, entry_patterns – поля ввода текста и образцов для 6.1.
- entry_wildcard_text, entry_wildcard_pattern, entry_wildcard_char – поля ввода для 6.2.
- text_intermediate, text_wildcard_intermediate (ctk.CTkTextbox) – области вывода промежуточных данных.
- canvas (tk.Canvas) – холст для визуализации автомата.
- automaton (AhoCorasick) – экземпляр автомата 6.1.
- wildcard_automaton (WildcardAhoCorasick) – экземпляр автомата 6.2.
- viz_source (ctk.StringVar) – текущий выбор в выпадающем меню визуализации.

Основные функции.

6.1 (Консоль и интерфейс)

- Node.init() – конструктор вершины бора, инициализирует поля.

- AhoCorasick.init() – конструктор автомата, создаёт корень.
- add_pattern(pattern, pattern_num) – добавляет образец в бор, создавая вершины и переходы; в терминальной вершине сохраняет номер образца.
- build_automaton() – строит суффиксные и терминальные ссылки методом обхода в ширину (BFS).
- search(text) / search_with_steps(text) – выполняет поиск всех вхождений образцов; в GUI-версии дополнительно сохраняет пошаговую трассировку.
- get_automaton_description() – возвращает текстовое описание автомата.
- get_transitions_description() – возвращает описание переходов бора.

6.2

- Node.init() – конструктор вершины бора сегментов.
- AhoCorasick.init() – конструктор автомата, создаёт корень.
- add_segment(segment, segment_id) – добавляет сегмент в бор; в терминальной вершине сохраняет ID сегмента.
- build_automaton() – строит суффиксные и сжатые суффиксные ссылки обходом в ширину.
- search_and_count(text, m, segments_info) – ищет сегменты в тексте, заполняет массив голосов C и возвращает список позиций вхождений.
- solve() – точка входа: читает входные данные, режет шаблон на сегменты, строит автомат, запускает поиск и выводит результаты.

Вариант 7 (GUI):

- KmpApp.init() – конструктор главного окна: создаёт вкладки и атрибуты automaton, wildcard_automaton, viz_source.
- setup_aho_corasick_tab() – настройка вкладки «Ахо-Корасик».
- setup_wildcard_tab() – настройка вкладки «Поиск с джокером».
- setup_visualization_tab() – настройка вкладки визуализации с выпадающим меню выбора автомата.

- build_automaton_gui() – обработчик построения автомата 6.1.
- search_patterns() – обработчик поиска 6.1 с пошаговым выводом.
- search_with_wildcard() – обработчик поиска 6.2: режет шаблон на сегменты, строит автомат WildcardAhoCorasick, запускает поиск с трассировкой и выводит все этапы.
- _get_current_automaton() – вспомогательный метод, возвращает выбранный пользователем автомат и его тип (aho или wildcard).
- draw_automaton() – универсальная отрисовка выбранного автомата на холсте.

Описание алгоритма

Работа программы начинается с инициализации: считывается текст T и количество образцов n, затем считываются сами образцы $p_1$, ..., $p$□.

Этап 1: Последовательно добавляем каждый образец: идём от корня по символам, при отсутствии перехода создаём новую вершину. В конечной вершине сохраняем номер образца в списке patterns

Этап 2: Построение суффиксных и терминальных ссылок

1. Для сыновей корня суффиксная ссылка ведёт в корень, output_link = None
2. Для каждой вершины в очереди и каждого её ребёнка:
    a. идём по суффиксным ссылкам родителя, пока не найдём вершину с переходом по символу ребёнка или не дойдём до корня;
    b. если переход найден – суффиксная ссылка ребёнка ведёт в эту дочернюю вершину, иначе – в корень;
    c. терминальная ссылка: если суффиксная ссылка содержит паттерны – берём её, иначе – output_link от неё.

Этап 3: Поиск вхождений

1. Пока нет перехода по символу – откатываемся по суффиксным ссылкам.
2. Переходим по ребру или остаёмся в корне.
3. Идём по output_link от текущей вершины до корня, собирая все найденные образцы.

6.2

Задача сводится к множественному поиску: шаблон разрезается на бездджокерные сегменты, для них строится один автомат Ахо-Корасик.

1. Фаза 1. Разбиение шаблона на сегменты. Идём по шаблону:
   a. копим символы в текущий сегмент;
   b. при встрече джокера: если сегмент не пуст, сохраняем пару (offset, строка), где offset – позиция первого символа сегмента; обновляем offset = i + 1;
   c. после обхода сохраняем последний незакрытый сегмент.
2. Этап 2. Построение автомата. Каждый сегмент добавляется в бор. В терминальной вершине сохраняется ID сегмента. Обходом в ширину строятся суффиксные и сжатые суффиксные ссылки.
3. Этап 3. Поиск и подсчёт голосов. Идём по тексту, поддерживая текущее состояние:
   a. При отсутствии перехода – откат по суффиксным ссылкам.
   b. Переходим в новую вершину.
   c. Идём по цепочке output_link от текущей вершины до корня. Для каждого seg_id в segment_ids:
   d. start_pos = i - seg_len + 1 – начало сегмента в тексте;
   e. template_start = start_pos - offset – предполагаемое начало всего шаблона;
   f. если старт не выходит за границы, C[template_start] += 1.

4. Этап 4. Сбор результатов. num_segments = len(segments_info). Позиция i – вхождение тогда и только тогда, когда C[i] == num_segments. Переводим в 1-based нумерацию и выводим.

Вариант 7 (Графическое представление автомата)

GUI не меняет алгоритмы, а добавляет к ним:

1. Ввод данных через поля на соответствующих вкладках.
2. Пошаговую трассировку. При поиске 6.1 для каждого символа текста сохраняются: состояние до, список переходов (включая откаты по суффиксным ссылкам), найденные образцы, состояние после. При поиске 6.2 – то же плюс «голоса» за стартовые позиции шаблона и снимок массива C.
3. Визуализацию автомата. На холсте рисуются вершины, переходы бора (сплошные стрелки с подписями символов) и суффиксные ссылки (пунктирные красные линии). Терминальные вершины выделяются голубой заливкой и красной рамкой с подписью P[..] (для 6.1) или S[..] (для 6.2).
4. Выбор автомата для отрисовки через выпадающее меню на вкладке визуализации.

Оптимизация алгоритма

Для задачи 6.1 (Ахо-Корасик):

1. Построение бора за $O(\Sigma|p_i|)$. Каждый символ каждого образца обрабатывается один раз при добавлении в бор.
2. Построение суффиксных ссылок за $O(\Sigma|p_i|)$. Обход в ширину гарантирует, что каждая вершина обрабатывается один раз. Амортизированная сложность подъёма по суффиксным ссылкам ограничена глубиной бора.

3. Поиск за O(|T| + количество_вхождений). Каждый символ текста обрабатывается один раз. Подъём по терминальным ссылкам выполняется только при нахождении образцов.
4. Использование словаря для переходов. Словарь Python обеспечивает O(1) в среднем для проверки наличия перехода и получения дочерней вершины.
5. Предварительное хранение образцов. Список patterns позволяет быстро получать длину образца для вычисления позиции вхождения.

Для задачи 6.2 с джокером:

1. Сведение к Ахо-Корасик вместо прямого перебора: сложность падает с O(|T| × |P|) до O(|T| + Σ|pi| + K).
2. Ленивые переходы – функция go не хранится в таблице, а вычисляется откатами по суффиксным ссылкам; амортизированно это O(|T|) на весь поиск и экономит память.
3. Сжатые суффиксные ссылки – позволяют за один шаг перечислять все сегменты, заканчивающиеся в текущей позиции.
4. Массив голосов C – вместо проверки каждой позиции отдельно.

Для Варианта 7 (Графический интерфейс):

1. Разделение интерфейса на вкладки через CTkTabview.
2. Пошаговый вывод промежуточных данных – соответствует требованиям лабораторной работы.
3. Цветовая индикация результатов: зелёный при успехе, красный при отсутствии.
4. Модульная архитектура – алгоритмические классы отделены от методов GUI.
5. Универсальная визуализация – один метод draw_automaton работает с обоими автоматами.

Оценка сложности

6.1

Теоретическая сложность:

- Временная сложность: O(|T| + Σ|$p_i$| + количество_вхождений).
    - Построение бора: O(Σ|p|), где Σ|p| – суммарная длина всех образцов.
    - Построение суффиксных ссылок: O(Σ|p|).
    - Поиск в тексте: O(|T| + количество_вхождений).
    - Сортировка результатов: O(K log K), где K – количество вхождений.
- Пространственная сложность: O(Σ|p| × |алфавит|).
    - Бор содержит не более Σ|$p_i$| + 1 вершин.
    - Каждая вершина хранит словарь переходов (в худшем случае |алфавит| элементов).
    - Суффиксные и терминальные ссылки: O(Σ|p|).

Практическая сложность:

Алгоритм Ахо-Корасик эффективно обрабатывает тексты длиной до 100 000 символов и до 3 000 образцов длиной до 75 символов. Суммарная длина образцов не превышает 225 000 символов, что обеспечивает построение автомата за короткое время.

6.2

Теоретическая сложность:

- Временная: O(|T| + Σ|pi| + K), где Σ|pi| – суммарная длина сегментов (не превышает длину шаблона, то есть ≤ 40). Разбиение шаблона – O(|P|), построение бора и ссылок – O(Σ|pi|), поиск – O(|T| + K) амортизированно.
- Пространственная: O(Σ|pi| × |алфавит|) на бор, O(|T|) на массив C, O(num_segments) на segments_info.

Практическая сложность:

При |T| ≤ 100 000 и |P| ≤ 40 суммарная длина сегментов мала, поэтому автомат строится мгновенно, а поиск проходит за один проход по тексту.

Вариант 7 (Графическое представление автомата):

- Временная сложность построения графа: O(V + E), где V – количество вершин, E – количество рёбер.
- Пространственная сложность: O(V + E).
- Отрисовка: O(V + E).

Оценка по памяти

6.1

- Бор: $O(\Sigma|p_i| \times |\text{алфавит}|)$ – при $\Sigma|p| = 225\,000$ и $|\text{алфавит}| = 5$ занимает порядка нескольких мегабайт.
- Суффиксные и терминальные ссылки: $O(\Sigma|p_i|)$.
- Список образцов: $O(\Sigma|p|)$.
- Результаты поиска: O(K), где K – количество вхождений.

Задача 6.2 с джокером:

- Бор сегментов: $O(\Sigma|pi| \times |\text{алфавит}|)$ – существенно меньше, чем для 6.1, так как сегментов мало и они короткие.
- Суффиксные и сжатые суффиксные ссылки: $O(\Sigma|pi|)$.
- Массив C: O(|T|).
- segments_info и seg_checks: O(num_segments).
- Трассировка (только в GUI): O(|T| × num_segments) в худшем случае.

Вариант 7 (Графическое представление):

- Структуры данных графа: O(V + E).
- Буфер отрисовки: зависит от библиотеки; для автоматов с несколькими тысячами вершин – десятки мегабайт.

**Тестирование**

Экспериментальные результаты подтверждают корректность работы алгоритма на различных тестовых случаях, включая:

Таблица 1: Тестирование программы 6.1

| Входные данные | Выходные данные |
|---|---|
| NTAG<br>TAGT,TAG,T | 2 2, 2 3 |
| ababab<br>ab,ba,a | 1 1, 1 3, 2 2, 3 1, 3 3, 4 2, 5 1, 5 3, 6 2 |
| acdef<br>abc, def, xyz | 1 1, 4 2 |
| aaaa<br>a, aa, aaa | 1 1, 1 2, 1 3, 2 1, 2 2, 3 1, 3 3, 4 1, 4 2, 5 1 |
| abc<br>abc | 1 1 |

Таблица 2: Тестирование программы 6.2

| Входные данные | Выходные данные |
|---|---|
| ACTANCA<br>A$A<br>$ | 1 |
| xabuccsbababcax | 2, 9 |
| ab??c?<br>? | 0 |
| aaaa<br>a$a<br>$ | 1,2,3 |

**Вывод**

В ходе выполнения лабораторной работы был реализован алгоритм Ахо-Корасик для точного поиска множества образцов в тексте. Алгоритм включает построение бора, вычисление суффиксных и сжатых суффиксных (output) ссылок, а также поиск всех вхождений с сортировкой результатов. Для задачи поиска образца с джокером был применён тот же аппарат: шаблон разбивается на сегменты без джокеров, для них строится автомат Ахо-Корасик, а найденные сегменты «голосуют» за возможные стартовые позиции шаблона через массив C. Такой подход снижает временную сложность задачи с джокером с O(|T| × |P|) до O(|T| + Σ|pi| + K). Для индивидуального варианта 7 был разработан графический интерфейс на базе библиотеки CustomTkinter. Интерфейс обеспечивает интерактивный ввод данных, вывод промежуточных данных (построение бора, описание вершин и переходов, пошаговый процесс поиска с трассировкой голосов) и наглядное графическое представление автоматов. Реализована универсальная визуализация: пользователь может переключаться между автоматом задачи 6.1 и автоматом сегментов задачи 6.2 через выпадающее меню, при этом терминальные вершины подписываются по-разному (P для паттернов, S для сегментов). Теоретическая сложность реализованных алгоритмов составляет O(|T| + Σ|pi| + K) по времени и O(Σ|pi| × |алфавит|) по памяти для обеих задач.